Machine Learning System Design Interview
MY CURRICULUM

01 Introduction and Overview
02 Visual Search System
03 Google Street View Blurring System
04 YouTube Video Search
05 Ad Click Prediction System
06 Video Recommendation System
07 Event Recommendation System
08 ETA Prediction for Ride-sharing Services
09 Nearby Friends and Proximity Service
10 Recommendation News Feed
11 Photos Top Ranking Service

03
Google Street View Blurring System

Google Street View (GSV) is a technology in Google Maps that provides street-level interactive panoramas of many cities' landmarks around the world. It allows users to virtually explore these cities by navigating through their streets. Users and business owners can upload their own photos to this viewport, as design a blurring system similar to Google Street View.

Figure 1: A Street View image with blurred license plates

Clarifying Requirements

Here's a typical interview between a candidate and the interviewer.

Candidate: Is it fair to say the business objective of the system is to protect user privacy?
Interviewer: Yes.

Candidate: We want to design a system that detects all human faces and license plates in Street View images and blurs them before displaying them to users. Is that correct? Can I assume users can report images that are not correctly blurred?
Interviewer: That is correct.

Candidate: Can we focus on automatic blurring?
Interviewer: Yes, automatic blurring is a hard sub-problem. Human faces and license plates are manually annotated in Street View images.

Candidate: The dataset may be unbalanced. Some images contain faces, while others may cause a bias towards certain human ethnicities. Is it fair to assume we have a balanced dataset?
Interviewer: Good point. Unfortunately, that is true. We focus on fairness and bias later.

Candidate: Any, understanding is that latency is not a big concern, as the system can detect objects and then blurs offline. Is that correct?
Interviewer: Latency is not a big concern. Blurring should happen within a few minutes after a new image is uploaded.

Candidate: Do we have a dedicated human labeling team? If so, what is their accuracy? Can we assume human labelers are perfect?
Interviewer: Yes, we have a dedicated human labeling team. Their accuracy is very high, but they occasionally make mistakes.

Candidate: Any, understanding is that blurring is not a big concern, as the system can detect objects and then blurs offline. Is that correct?
Interviewer: Latency is not a big concern. Blurring should happen within a few minutes after a new image is uploaded.

Candidate: Do we have a dedicated human labeling team? If so, what is their accuracy? Can we assume human labelers are perfect?
Interviewer: Good point. Unfortunately, that is true. We focus on fairness and bias later.

Candidate: Any, understanding is that latency is not a big concern, as the system can detect objects and then blurs offline. Is that correct?
Interviewer: Latency is not a big concern. Blurring should happen within a few minutes after a new image is uploaded.

Candidate: Do we have a dedicated human labeling team? If so, what is their accuracy? Can we assume human labelers are perfect?
Interviewer: Good point. Unfortunately, that is true. We focus on fairness and bias later.

Frame the Problem as an ML Task

In this section, we frame the problem as an ML task.

Defining the ML objective

The business objective of the system is to protect user privacy by blurring visible license plates and human faces in Street View images. Broad definitions can be ambiguous, so we need to define the objective, so we need to translate it into an ML objective that an ML system can optimize. The most common ML objective for computer vision is minimizing cross-entropy loss. However, this is a proxy objective, and minimizing cross-entropy loss does not ensure the objective is blurring the images in a way.

Throughout this chapter, we use "objects" instead of "human faces and license plates" for convenience.

Specifying the system's input and output

The input of an object detection model is an image with zero or multiple objects at different locations within it. The model detects those objects and outputs their locations. Figure 2.1 shows an object detection system, along with its input and output.

Figure 2.1: Object detection system's input and output

Choosing the right ML category

In general, an object detection system has two responsibilities:

- Predicting the number of each object in the image
- Predicting the class of each bounding box (e.g., dog, cat, etc.)

The first task is a regression problem since the location can be represented by (x, y) coordinates, which are numeric values. The second task is a multi-class classification problem.

Traditionally, object detection architectures are divided into one-stage and two-stage networks. Recently, Transformer-based approaches have emerged, but we focus on the two-stage and one-stage approaches since they are more popular and mature than vision transformer-based approaches.

Two-stage networks

As the name implies, two-separate models are used in two-stage networks:

1. **Region proposal network (RPN):** Looks at an image and proposes candidate regions that are likely to be objects.
2. **Classifier:** processes each proposed region and classifies it into an object class.

Figure 2.2 shows these two stages.

Figure 2.2: Two-stage network

Commonly used two-stage networks include: R-CNN [2], and Mask R-CNN [3].

One-stage networks

In these networks, both stages are combined. Using a single network, bounding boxes and object classes are predicted simultaneously, without explicit detection of region proposals. Figure 2.3 shows a one-stage network.

Figure 2.3: One-stage network

Commonly used one-stage networks include: YOLO [4] and SSD [5] architectures.

One-stage vs. two-stage

Two-stage networks comprise two components that run separatedly, so they are usually slower, but more accurate.

In general, the trade-off between speed and accuracy is a business decision. Some applications, such as autonomous driving, require real-time speed because the training cost necessitates. So, for this exercise, we start with a two-stage network. When training does increase in complexity, we switch to the faster option, we can switch to a one-stage network.

Data Preparation

Data engineering

In the introduction chapter, we discussed data engineering fundamentals. Additionally, it's usually a good idea to discuss the specific data solutions for the task at hand. For this problem, we have the following data available:

- Annotated dataset
- Street View images

Let's discuss each in more detail.

Annotated dataset

Based on the requirements, we have 7 million annotated images. Each image has a list of bounding boxes and associated object classes. Table 2.1 shows data samples from this dataset.

image path	Objects	Bounding boxes
/data/image1.jpg	human face	[10,20,30,35]
/data/image2.jpg	human face	[132,340,45,70]
/data/image2.jpg	license plate	[300,95,20,10]
/data/image3.jpg	human face	[170,290,30,60]
/data/image3.jpg	license plate	[400,95,20,10]
/data/image3.jpg	human face	[30,40,30,60]

Table 2.1: A few data points from the annotated dataset

Each bounding box is a list of numbers: top-left x and y coordinates, followed by the box's width and height of the object.

Street View images

These are the Street View images collected by the data sourcing team. The ML system processes these images to detect human faces and license plates. Table 2.2 shows this metadata of the image.

image path	Location (lat, long)	Pitch, Yaw, Roll	Timestamp
/city/image1.jpg	37.0103037 -122.4448956	-25,35,0	1640170491
/city/image2.jpg	37.7749295, -122.4194155	0,0,0	1640170491
/city/image3.jpg	37.0103037, -121.9836956	-15,-20,0	1640170750

Table 2.2: Metadata of Street View images

Feature engineering

During feature engineering, we find good, dense features. Such as detecting edges and textures, rather than raw pixels, which are dense but noisy. Feature engineering is a powerful technique, and it's not a lost art. Even in the deep learning era, it's still a valuable technique.

Data augmentation

Data augmentation is a cheap way to generate more training data. It's a powerful technique that can significantly improve model performance. Some popular data augmentation techniques are:

- Random crop
- Random translation
- Random rotation
- Random scale translation
- Random color jitter
- Changing brightness, saturation, or contrast

Figure 2.4 shows an image with various data augmentation techniques applied to it.

Figure 2.4: Various data augmentation techniques applied to an image

Model Development

Model selection

As mentioned in the Frame the Problem as an ML Task section, we opted for two-stage networks. Figure 2.7 shows a typical two-stage architecture.

Figure 2.7: Two-stage object detection network

Region Proposal Network (RPN)

RPN proposes candidate regions that may contain objects. It uses neural networks as its architecture and takes the feature map produced by convolutional layers and proposes candidate regions for the classifier.

Classifier

The classifier determines the object class of each candidate region. It takes the feature map and the proposed candidate regions as input and predicts the object class for each region. The classifier is usually based on neural networks.

In VGG, a deep-learning network, you generally are expected to discuss the number of layers and the hidden dimensions.

Model training

The process of training a neural network usually involves three steps: forward propagation, loss calculation, and backpropagation. Modern deep-learning libraries can be familiar with these steps but for more information, see [10]. In this section, we discuss loss functions and how to calculate the loss for our system.

An object detection system needs to optimize two tasks at the same time. The first is classifying objects in the image. Cross-entropy is a high-value with the ground-truth bounding boxes. This is a regression task. Second, the predicted probabilities for each object class should be accurate. This is a classification task. Let's define a loss function for each task.

Regression loss

This loss measures how aligned the predicted bounding boxes are with the ground-truth bounding boxes. We use Huber loss, a robust loss function, such as Mean Squared Error (MSE) [11], and denote it by L_{box}.

$$L_{box} = \frac{1}{N}\sum_{i=1}^{N} \begin{cases} \frac{1}{2}(y_i - \hat{y}_i)^2 & \text{if } |y_i - \hat{y}_i| < \delta \\ \delta\,|y_i - \hat{y}_i| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases}$$

Where:

- N: total number of predictions
- y_i: ground-truth top-left x coordinate
- $\hat{y}_i$: predicted top-left x coordinate
- y_i: ground-truth top-left y coordinate
- $\hat{y}_i$: predicted top-left y coordinate
- y_i: ground-truth width
- $\hat{y}_i$: predicted width
- y_i: ground-truth height
- $\hat{y}_i$: predicted height

Classification loss

The classification loss accounts for the predicted probabilities per for each detected object. Here, we use a standard classification loss, such as log loss (crossentropy) [12] and denote it by L_{cls}.

$$L_{cls} = -\sum_{c=1}^{C} y_{o,c} \log(p_{o,c})$$

Where:

- M: total number of detections/bounding boxes
- C: total number of classes
- $y_{o,c}$: ground-truth label for detection o
- $p_{o,c}$: predicted class label for detection o

Similarly, a hyperparameter is needed to balance the model's overall performance, so combine the classification loss and regression loss with a hyperparameter λ:

$$L = L_{cls} + \lambda L_{box}$$

Evaluation

During an interview, it is crucial to discuss how to evaluate an ML system. The interviewer usually wants to know which metrics you'll choose and why. This section describes two object detection metrics you usually evaluated, and then selects important metrics for our Street View blurring system.

An object detection model usually has two metrics for different objects in an image. It measures the overall performance of the model, so evaluate each object separately and then average the results.

Figure 3.4 shows the output of an object detection model. It draws boxes around both the ground truth and detected bounding boxes. As shown, the model detected 5 bounding boxes, while we only have two instances of the object.

Figure 3.4: Ground truth and detected bounding boxes

When a predicted bounding box overlapped correct? To answer this question, we need to understand the definition of intersection over union.

Intersection Over Union (IOU)

IOU measures the overlap between two bounding boxes. Figure 3.5 shows a visual representation of IOU.

Figure 3.5: IOU formula

IOU determines whether a detected bounding box is correct. An IOU of 1 is ideal, indicating the detected bounding box and the ground truth bounding box are fully aligned. In practice, it's rare to see an IOU of 1. A higher IOU means the predicted bounding box is more accurate. An IOU threshold is usually used to determine whether a detected bounding box is correct (true positive) or incorrect (false positive). For example, if the IOU threshold is 0.5, any detection that has an overlap of at 0.5 or higher with the ground-truth bounding box is a correct detection.

Now we know what IOU is and how to determine correct and incorrect bounding box predictions, let's discuss metrics for whole evaluation.

Offline metrics

Model development is an iterative process. We use offline metrics to quickly evaluate the performance of newly developed models. Here are some metrics that might be useful for the object detection system.

- Precision
- Recall
- Mean average precision

Precision

This is the fraction of correct detections among all detections across all images. A high precision value shows the system's detections are more reliable.

Figure 3.6: Ground truth bounding boxes and detected bounding boxes

Let's calculate precision for three different IOU thresholds: 0.7, 0.5, and 0.1.

$$\text{Precision}_{IOU=0.7} = \frac{\text{Correct detections}_{IOU=0.7}}{\text{Total detections}} = \frac{2}{5} = 0.4$$

$$\text{Precision}_{IOU=0.5} = \frac{\text{Correct detections}_{IOU=0.5}}{\text{Total detections}} = \frac{3}{5} = 0.6$$

$$\text{Precision}_{IOU=0.1} = \frac{\text{Correct detections}_{IOU=0.1}}{\text{Total detections}} = \frac{4}{5} = 0.8$$

- **IOU threshold:** 0.7 This time, we have two correct detections.
- **IOU threshold:** 0.5 This time, we have three correct detections.
- **IOU threshold:** 0.1 This time, we have four correct detections.

Recall

According to the requirements, the system needs to protect the privacy of individuals. One way to measure this is to count the number of non-blurred objects. We can also use the recall metric to measure the system's performance. Recall is the percentage of correctly detected images. Other metrics that measure false and fairness are critical. For example, we want to ban human faces equally well across different race and gender groups. But mentioning this, is beyond the requirements. It is out of the scope.

To calculate the evaluation metrics, we use mAP and AP as our offline metrics. mAP measures the overall precision of the model, while AP gives an insight into the precision of the model of particular classes. The main metric of the entire evaluation is "mAP" (mean AP).

Serving

In this section, we talk about a common pattern that many ML systems deploy: overlapping bounding boxes. We also discuss a simple serving design.

Overlapping bounding boxes

Object detection models usually predict several bounding boxes per an image, it is very common to see bounding boxes overlap. This is because the RPN usually proposes various highly overlapping bounding boxes around each object. It is important to narrow down these bounding boxes to a single bounding box per object. Non-maximum suppression (NMS) [13], can's surprise how it works.

NMS is a post-processing algorithm designed to select the most appropriate bounding boxes. It keeps highly confident bounding boxes and removes overlapping bounding boxes. Figure 3.7 shows an example.

Figure 3.7: Post-processing with NMS

NMS is a commonly used algorithm in ML system design interviews, so you're encouraged to have a good understanding of it [14].

ML system design

As illustrated in Figure 3.8, we propose an ML system design for the blurring system.

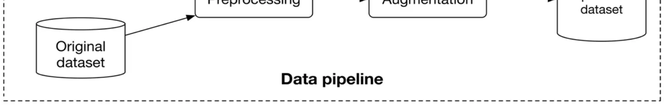

Figure 3.12: ML system design

Let's examine each pipeline in more detail.

Batch prediction pipeline

Based on the requirements gathered, latency is not a big concern because we can display existing images to users while new ones are being processed. Since instant results are not required, we can utilize batch prediction and precompute the object detection results.

Preprocessing Raw images are preprocessed by this component. This section does not discuss the preprocess operations as we have already discussed them in the feature engineering section.

Blurring service This performs the following operations on a Street View image:

1. Provides a list of objects detected in the image.
2. Refines the list of detected objects using the NMS component.
3. Blurs detected objects.
4. Stores the blurred image in object storage (Blurred Street View images).

Note that the preprocessing and blurring services are separate in the design. The reason is preprocessing images tends to be a CPU-bound process, whereas blurring service relies on GPU. Separating these services has two benefits:

- Scale the services independently based on the workload each receives.
- Better utilization of CPU and GPU resources.

Data pipeline

This pipeline is responsible for processing users' reports, generating new training data, and preparing training data to be used by the model. Data pipeline components are mostly self-explanatory. Hard negative mining is the only component that needs more explanation.

Hard negative mining. Hard negatives are examples that are explicitly created as negatives out of incorrectly predicted examples, and then added to the training dataset. When we retrain the model on the updated training dataset, it should perform better.

Other Talking Points

If time allows, here are some additional points to discuss:

- How Transformer-based object detection architectures differ from one-stage or twostage models, and what are their pros and cons [18].
- Distributed training techniques to improve object detection on a larger dataset [19] [20].
- How General Data Protection Regulation (GDPR) in Europe may affect our system [21].
- Evaluate bias in face detection systems [22] [23].
- How to continuously fine-tune the model [24].
- How to use active learning [25] or human-in-the-loop ML [26] to select data points for training.

References

1. Google Street View. https://www.google.com/streetview.
2. DETR. https://github.com/facebookresearch/detr.
3. RCNN family. https://lilianweng.github.io/posts/2017-12-31-object-recognition-part-3.
4. Fast R-CNN paper. https://arxiv.org/pdf/1504.08083.pdf.
5. Faster R-CNN paper. https://arxiv.org/pdf/1506.01497.pdf.
6. YOLO family. https://pyimagesearch.com/2022/04/04/introduction-to-the-yolo-family.
7. SSD. https://jonathan-hui.medium.com/ssd-object-detection-single-shot-multibox-detector-for-real-time-processing-9bd8deac0e06.
8. Data augmentation techniques. https://www.kaggle.com/getting-started/190280.
9. CNN. https://en.wikipedia.org/wiki/Convolutional_neural_network.
10. Forward pass and backward pass. https://www.youtube.com/watch?v=qzPQ8cEsVK8.
11. MSE. https://en.wikipedia.org/wiki/Mean_squared_error.
12. Log loss. https://en.wikipedia.org/wiki/Cross_entropy.
13. Pascal VOC. http://host.robots.ox.ac.uk/pascal/VOC/voc2008/index.html.
14. COCO dataset evaluation. https://cocodataset.org/#detection-eval.
15. Object detection evaluation. https://github.com/rafaelpadilla/Object-Detection-Metrics.
16. NMS. https://en.wikipedia.org/wiki/NMS.
17. Pytorch implementation of NMS. https://learnopencv.com/non-maximum-suppression-theory-and-implementation-in-pytorch/.
18. Recent object detection models. https://viso.ai/deep-learning/object-detection/.
19. Distributed training in Tensorflow. https://www.tensorflow.org/guide/distributed_training.
20. Distributed training in Pytorch. https://pytorch.org/tutorials/beginner/dist_overview.html.
21. GDPR and ML. https://www.oreilly.com/radar/how-will-the-gdpr-impact-machine-learning.
22. Bias and fairness in face detection. http://sibgrapi.sid.inpe.br/col/sid.inpe.br/sibgrapi/2021/09.04.19.00/doc/103.pdf.
23. AI fairness. https://www.kaggle.com/code/alexisbcook/ai-fairness.
24. Continual learning. https://towardsdatascience.com/tag/fine-tuning/.
25. Active learning. https://en.wikipedia.org/wiki/Active_learning_(machine_learning).
26. Human-in-the-loop ML. https://arxiv.org/pdf/2108.00941.pdf.

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.